

EXPERIMENT – 01

Date: 11/09/2026

Title: Bank Account Management System using Classes and Objects

Objective / Aim

To design and implement a **Bank Account Management System** in Java using **classes and objects**, demonstrating object-oriented programming concepts such as attributes, methods, constructors, object creation, inheritance, method overriding, deposits, withdrawals, interest calculation, and fund transfers.

System Requirements

Hardware

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

Software

- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK 17 or JDK 21)
- Eclipse / IntelliJ IDEA / NetBeans / VS Code
- Command Prompt / Terminal

Algorithm / Logic Description

1. Create a base class named **Account**.
2. Define the following attributes in the Account class:
 - Account number
 - Account holder name
 - Balance
 - Account type
7. Define a constructor to initialize the account details.
8. Define the following methods in the Account class:
 - **deposit()** to add money to the account.
 - **withdraw()** to withdraw money when sufficient balance is available.
 - **transfer()** to transfer money from one account to another.
 - **displayAccountDetails()** to display account information.
13. Create a **SavingsAccount** class that inherits from Account.
14. Add **interestRate** as an additional attribute in SavingsAccount.
15. Define **calculateInterest()** to calculate interest based on the account balance.
16. Create a **CurrentAccount** class that inherits from Account.

17. Add **overdraftLimit** as an additional attribute in CurrentAccount.
18. Override the **withdraw()** method in CurrentAccount to allow withdrawal up to the permitted overdraft limit.
19. Create objects for SavingsAccount and CurrentAccount.
20. Perform deposits, withdrawals, interest calculation, and fund transfer operations.
21. Display the account details before and after the transactions.
22. Display the final balance of both accounts.

NOTE: SavingsAccount and CurrentAccount inherit from the Account class.

UML Class Diagram



Relationship: SavingsAccount and CurrentAccount inherit from the Account class.

Source Code

File Name: BankAccountDemo.java

```
class Account {

    protected String accountNumber;
    protected String holderName;
    protected double balance;
    protected String accountType;

    // Constructor
    public Account(String accountNumber, String holderName,
                   double balance, String accountType) {

        this.accountNumber = accountNumber;
        this.holderName = holderName;
        this.balance = balance;
        this.accountType = accountType;
    }

    // Deposit method
    public void deposit(double amount) {

        if (amount > 0) {
            balance = balance + amount;
            System.out.println("Deposited: Rs." + amount);
        } else {
            System.out.println("Invalid deposit amount.");
        }
    }

    // Withdraw method
    public boolean withdraw(double amount) {

        if (amount > 0 && amount <= balance) {
            balance = balance - amount;
            System.out.println("Withdrawn: Rs." + amount);
            return true;
        } else {
            System.out.println("Insufficient balance.");
            return false;
        }
    }

    // Transfer method
    public void transfer(Account target, double amount) {

        if (withdraw(amount)) {
            target.deposit(amount);

            System.out.println("Transferred: Rs." + amount
                               + " to " + target.accountNumber);
        } else {
            System.out.println("Transfer failed.");
        }
    }

    // Display account details
    public void displayAccountDetails() {

        System.out.println("Account Number : " + accountNumber);
        System.out.println("Holder Name : " + holderName);
        System.out.println("Account Type : " + accountType);
        System.out.printf("Balance : Rs.%.2f\n", balance);
    }
}

// Savings Account class
class SavingsAccount extends Account {

    private double interestRate;

    // Constructor
    public SavingsAccount(String accountNumber,
                          String holderName,
                          double balance,
                          double interestRate) {

        super(accountNumber, holderName, balance, "Savings");
    }
}
```

```

        this.interestRate = interestRate;
    }

    // Calculate interest
    public double calculateInterest() {

        return balance * interestRate / 100;
    }

    // Method overriding
    @Override
    public void displayAccountDetails() {

        super.displayAccountDetails();

        System.out.println("Interest Rate : " + interestRate + "%");

        System.out.printf("Interest      : Rs.%.2f%n",
            calculateInterest());
    }
}

// Current Account class
class CurrentAccount extends Account {

    private double overdraftLimit;

    // Constructor
    public CurrentAccount(String accountNumber,
        String holderName,
        double balance,
        double overdraftLimit) {

        super(accountNumber, holderName, balance, "Current");
        this.overdraftLimit = overdraftLimit;
    }

    // Method overriding
    @Override
    public boolean withdraw(double amount) {

        if (amount > 0 && amount <= balance + overdraftLimit) {

            balance = balance - amount;

            System.out.println("Withdrawn: Rs." + amount);

            return true;

        } else {

            System.out.println(
                "Withdrawal exceeds overdraft limit.");

            return false;
        }
    }

    // Method overriding
    @Override
    public void displayAccountDetails() {

        super.displayAccountDetails();

        System.out.printf("Overdraft Limit: Rs.%.2f%n",
            overdraftLimit);
    }
}

// Main class
public class BankAccountDemo {

    public static void main(String[] args) {

        // Creating SavingsAccount object
        SavingsAccount savings =
            new SavingsAccount(
                "SA101",
                "Rahul",

```

```

        10000,
        5.0
    );

    // Creating CurrentAccount object
    CurrentAccount current =
        new CurrentAccount(
            "CA201",
            "K.SESHU",
            15000,
            5000
        );

    // Savings Account
    System.out.println("===== SAVINGS ACCOUNT =====");

    savings.displayAccountDetails();

    System.out.println("\nDepositing Rs.2000...");

    savings.deposit(2000);

    System.out.println("\nWithdrawing Rs.1500...");

    savings.withdraw(1500);

    // Savings account after transactions
    System.out.println("\n===== AFTER TRANSACTIONS =====");

    savings.displayAccountDetails();

    // Current Account
    System.out.println("\n===== CURRENT ACCOUNT =====");

    current.displayAccountDetails();

    System.out.println("\nWithdrawing Rs.18000...");

    current.withdraw(18000);

    // Current account after withdrawal
    System.out.println("\n===== AFTER WITHDRAWAL =====");

    current.displayAccountDetails();

    // Transfer
    System.out.println("\n===== TRANSFER =====");

    System.out.println(
        "Transferring Rs.2000 from Savings to Current..."
    );

    savings.transfer(current, 2000);

    // Final account details
    System.out.println("\n===== FINAL ACCOUNT DETAILS =====");

    System.out.println("\nSavings Account:");

    savings.displayAccountDetails();

    System.out.println("\nCurrent Account:");

    current.displayAccountDetails();
}
}

```

Compilation and Execution

Save the program as:

```
BankAccountDemo.java
```

Compile:

```
javac BankAccountDemo.java
```

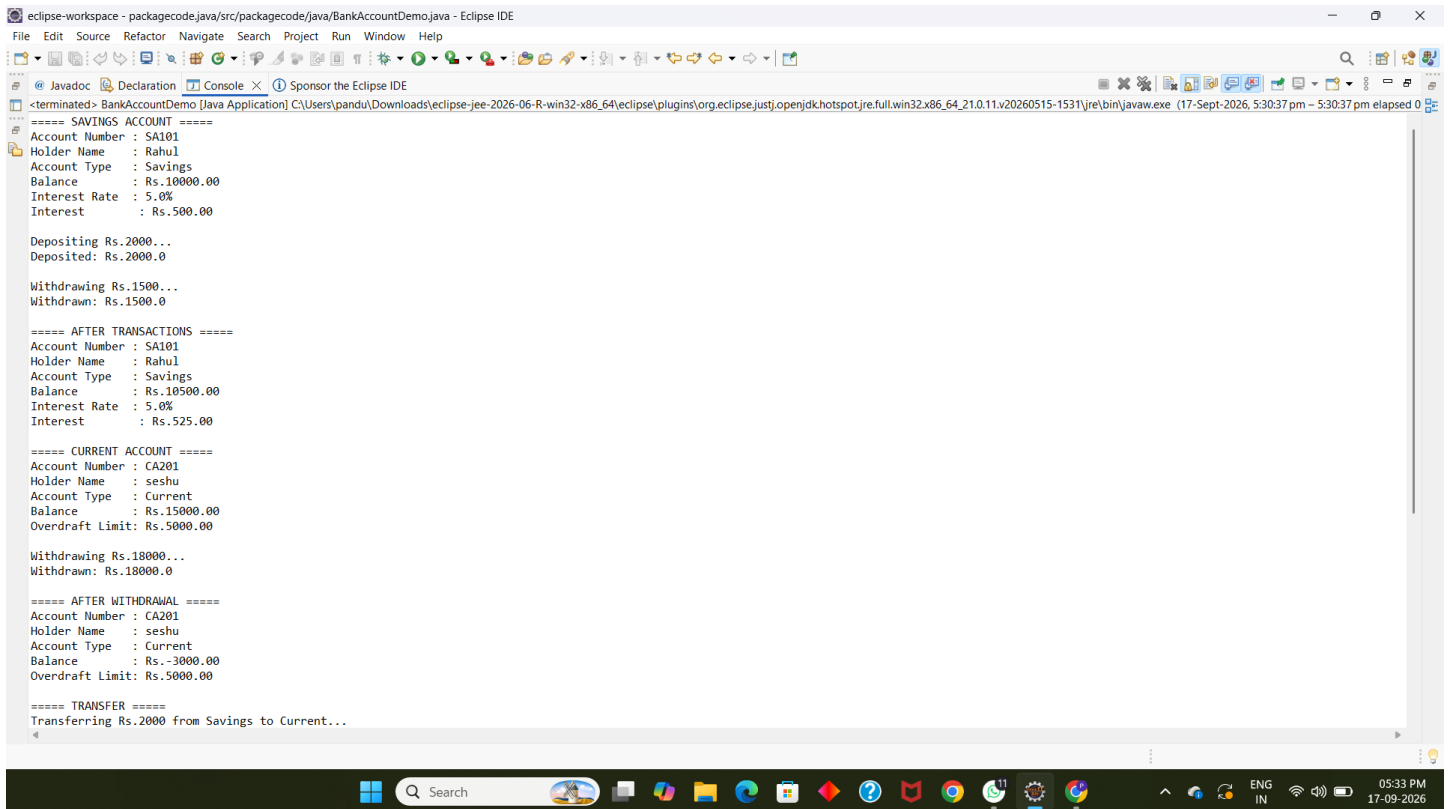
Run:

```
java BankAccountDemo
```

Result

The **Bank Account Management System** was successfully implemented in Java using classes and objects. The program demonstrates attributes, methods, constructors, object creation, inheritance, method overriding, deposits, withdrawals, interest calculation, fund transfers, and displaying account details successfully.

Output Screen Shot 1



The screenshot shows the Eclipse IDE interface with the console output of a Java application. The application is titled "BankAccountDemo [Java Application]" and is running on a Windows machine. The console output displays the following sequence of events:

```
<terminated> BankAccountDemo [Java Application] C:\Users\pandu\Downloads\eclipse-jee-2026-06-R-win32-x86_64\eclipse\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.11.v20260515-1531\jre\bin\javaw.exe (17-Sept-2026, 5:30:37 pm - 5:30:37 pm elapsed 0)

===== SAVINGS ACCOUNT =====
Account Number : SA101
Holder Name : Rahul
Account Type : Savings
Balance : Rs.10000.00
Interest Rate : 5.0%
Interest : Rs.500.00

Depositing Rs.2000...
Deposited: Rs.2000.0

Withdrawing Rs.1500...
Withdrawn: Rs.1500.0

===== AFTER TRANSACTIONS =====
Account Number : SA101
Holder Name : Rahul
Account Type : Savings
Balance : Rs.10500.00
Interest Rate : 5.0%
Interest : Rs.525.00

===== CURRENT ACCOUNT =====
Account Number : CA201
Holder Name : seshu
Account Type : Current
Balance : Rs.15000.00
Overdraft Limit: Rs.5000.00

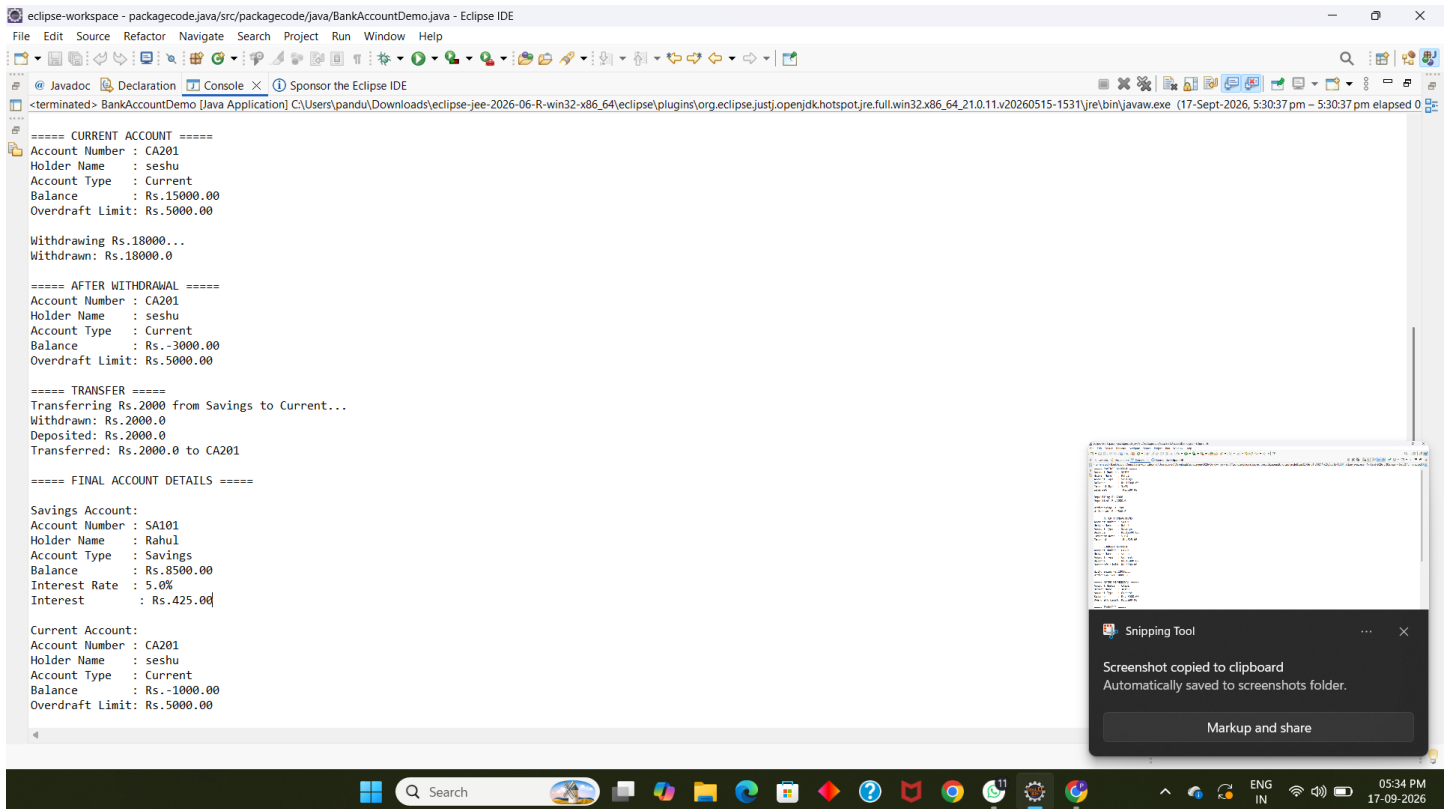
Withdrawing Rs.18000...
Withdrawn: Rs.18000.0

===== AFTER WITHDRAWAL =====
Account Number : CA201
Holder Name : seshu
Account Type : Current
Balance : Rs.-3000.00
Overdraft Limit: Rs.5000.00

===== TRANSFER =====
Transferring Rs.2000 from Savings to Current...
```

The bottom of the image shows the Windows taskbar with the search bar, task view button, and several application icons. The system clock indicates the time is 05:33 PM on 17-09-2026.

Output Screen Shot 2



```
eclipse-workspace - packagecode.java/src/packagecode/java/BankAccountDemo.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help

@ Javadoc Declaration Console x Sponsor the Eclipse IDE
<terminated> BankAccountDemo [Java Application] C:\Users\pandu\Downloads\eclipse-jee-2026-06-R-win32-x86_64\eclipse\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.21.0.11.v20260515-1531\jre\bin\javaw.exe (17-Sept-2026, 5:30:37 pm - 5:30:37 pm elapsed 0)

===== CURRENT ACCOUNT =====
Account Number : CA201
Holder Name : seshu
Account Type : Current
Balance : Rs.15000.00
Overdraft Limit: Rs.5000.00

Withdrawing Rs.18000...
Withdrawn: Rs.18000.0

===== AFTER WITHDRAWAL =====
Account Number : CA201
Holder Name : seshu
Account Type : Current
Balance : Rs.-3000.00
Overdraft Limit: Rs.5000.00

===== TRANSFER =====
Transferring Rs.2000 from Savings to Current...
Withdrawn: Rs.2000.0
Deposited: Rs.2000.0
Transferred: Rs.2000.0 to CA201

===== FINAL ACCOUNT DETAILS =====

Savings Account:
Account Number : SA101
Holder Name : Rahul
Account Type : Savings
Balance : Rs.8500.00
Interest Rate : 5.0%
Interest : Rs.425.00

Current Account:
Account Number : CA201
Holder Name : seshu
Account Type : Current
Balance : Rs.-1000.00
Overdraft Limit: Rs.5000.00
```

Snipping Tool

Screenshot copied to clipboard
Automatically saved to screenshots folder.

Markup and share

05:34 PM
17-09-2026